

CALIFORNIA POLYTECHNIC UNIVERSITY, POMONA

ECE 4990 - SECTION 1

WIRELESS & MOBILE SYSTEMS

GROUP 6 PROJECT REPORT

Ad-hoc Network of Environment-Sensing Microcomputers

Student

ETHAN MCKENDELL

BRYAN RAMILLANO

ANDREW RAVADAN

Professor

DR. TAMER OMAR

ABSTRACT

This project, Raspberry Pi Zero Ad Hoc Sensor Network, explores the implementation of a decentralized wireless mesh network using the B.A.T.M.A.N. (Better Approach To Mobile Adhoc Networking) protocol. The goal is to reliably transmit environmental data, specifically temperature and humidity readings, using a network of low-cost, low-power microcomputers, namely Raspberry Pi Zero 2Ws, as sensor nodes. In this architecture, each Pi functions as a peer in a fully distributed ad hoc network, with no central coordinator or router. This peer-to-peer approach allows the network to self-organize, self-heal, and dynamically adjust to changes such as node leaving or entering the network.

One of the key advantages of ad hoc networks is their scalability. As more Raspberry Pi nodes are added, the network grows organically without requiring infrastructure changes or reconfiguration. Each new node participates in both sensing and routing, increasing the network's coverage. BAT MAN enhances this scalability by handling route discovery and optimization, allowing data to traverse multiple hops through the network to reach its destination efficiently, even if the source and destination are not within direct wireless range of each other.

Our primary application for this ad hoc mesh network is in greenhouse monitoring. By placing Raspberry Pi Zero nodes throughout a greenhouse environment, we can collect distributed temperature and humidity data in real-time without needing Wi-Fi access points or wired infrastructure. The multi-hop capability of B.A.T.M.A.N. ensures that data from distant or obstructed areas can still reach a central node or gateway by "hopping" through intermediate nodes. This approach is especially beneficial in environments where wireless signals may be blocked by plants, structures, or equipment.

Beyond greenhouse monitoring, the same network architecture has broad applications in smart agriculture, environmental sensing, disaster recovery, mobile robotics, and any scenario requiring flexible, infrastructure-free connectivity. The combination of B.A.T.M.A.N.'s intelligent routing and the affordability of Raspberry Pi Zeros offers a powerful and practical solution for deploying large-scale sensor networks in dynamic or remote environments.

Contents

1	INTRODUCTION	3
2	PROJECT OBJECTIVES	4
3	PROJECT METHODOLOGY	5
3.1	Materials Required	5
3.2	Overall System Flow for Receivers and Senders	6
3.3	Setting up Pi Nodes	7
3.3.1	Setting up Sender Pi Node	9
3.3.2	Setting up Receiver Pi Node	11
3.4	End Result	13
4	PROJECT RESULTS	14
4.1	Sensor Node & Receiver Node (Objective #1 #2)	14
4.2	Multi-hop Routing (Objective #3)	15
4.3	Battery Life Efficiency (Objective #4)	16
5	SUMMARY OF CHALLENGES AND HOW THEY WERE SOLVED	17
5.1	Switching from IBSS Networking to B.A.T.M.A.N	17
5.2	Public WiFi vs Private WiFi for setting up Pis	18
5.3	Saving Costs	19
6	CONCLUSIONS	20
7	REFERENCES	21

1 INTRODUCTION

Ad Hoc Networks Such networks are wireless networks that lack fixed infrastructure, meaning that they lack traditional technologies like routers and access points. Each device in the network communicates directly with other devices, and contributes to forwarding data. Each device in the network communicates directly with other devices, and contributes to forwarding data.

Usage of Low-Cost Microcomputers They are small and affordable computers that contain the essential parts of a computer, with a CPU (microprocessor), IO, and storage. Microcomputers are designed to be used for either performing general tasks or dedicated to performing a single task. Due to their compact size, they typically see their use in embedded systems, prototyping, and Internet of Things applications.

Practical Uses of Ad Hoc Networks with Microcomputers Microcomputers are ideal for Ad Hoc Networks due to their low cost, portability, and ease of programming. They are already equipped with the necessary interfaces to allow for peer-to-peer communication like what will be used in our Ad Hoc Network. These characteristics allow us to create an Ad Hoc Network that can be easily disassembled and modified to allow for more complex and interesting applications.

2 PROJECT OBJECTIVES

For this project, there is 4 goals this project will achieve:

1. Create a **sensor node** to capture temperature and humidity data and send it over an ad hoc network.
2. Create a **receiver node** to receive temperature and humidity data and displaying it.
3. Implement multi-hop routing to enable sensor nodes to transmit environmental data to the receiver node, regardless of distance.
4. Ensure each **sensor node** is able to survive up to 5 hours on an external battery.

3 PROJECT METHODOLOGY

3.1 Materials Required

- 6 Raspberry Pi Zero 2W
- 5 Raspberry Pi's will be used for the senders and 1 Raspberry Pi will be used for the receiver.
- 5 DHT11
- 6 MicroSD Cards with Raspberry Pi Lite OS Installed (minimum 4GB)
- 6 USB Micro to USB A
- 6 Power Banks $\geq 1600mAh$
- 1 Monitor

The monitor has two purpose. First is to type in the sudo command lines to install all dependencies because once ad hoc mode is activated, any SSH connection will be lost, so having desktop access ensures that the user can still visually control the Raspberry Pi. Secondly, it is needed to display the data collected by the receiver.

- 1 MicroHDMI to HDMI (or input that the monitor being used has)
- 1 USB Keyboard
- Necessary jumper cables and bread boards to connect DHT11 to sender Pis.

3.2 Overall System Flow for Receivers and Senders

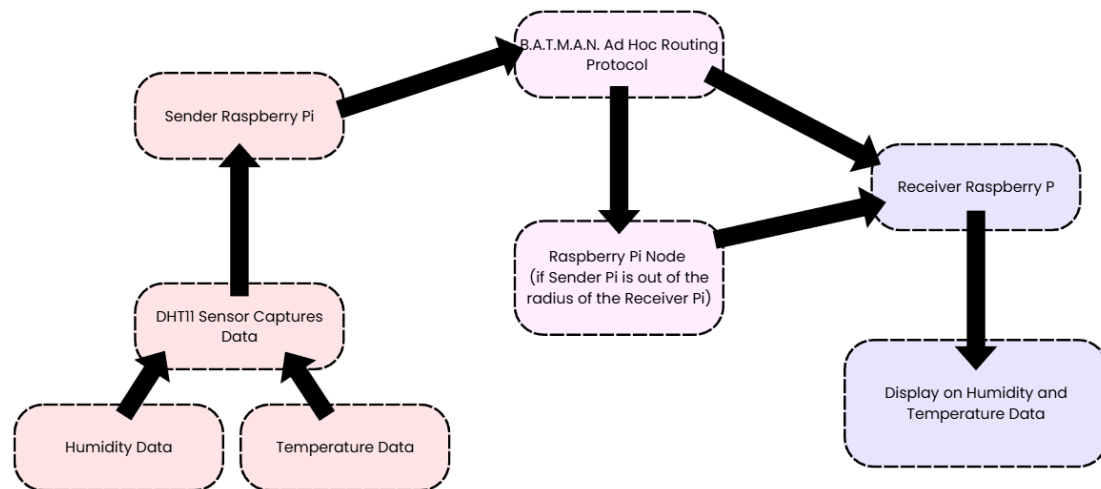


Figure 1: System diagram for Ad-Hoc Network of Environment-Sensing Microcomputers.

3.3 Setting up Pi Nodes

In order to set up each sender or receiver pi:

1. Insert the MicroSD card with Raspberry Pi OS Lite installed into the Raspberry Pi Zero 2W (or any other Raspberry Pi model being used), and connect the Pi to a monitor and a USB keyboard.

2. Set up the internet connection:

```
sudo raspi-config #open settings and connect to internet.
```

To test if the internet connection is good, ping a website. If there is data recieved, that means there is data collected

```
ping -c 3 google.com #or any website
```

3. Install python 3 , git, and pip3. These dependencies are nessesary to access Github and download the code necessary for this project to run

```
python3 --version #check to see if python3 is installed
sudo apt install python3 -y #install python3
```

```
git --version #check to see if git is installed
sudo apt install git -y #install git
```

```
pip3 --version #check to see if pip3 is installed
sudo apt install python3-pip -y #install pip3
```

4. Clone the git repository where the code is located onto the Pi:
<https://github.com/MathAlpha24/Raspberry-Pi-Zero-Adhoc-Sensor-Network.git>

```
git clone "git link above"
cd Raspberry-Pi-Zero-Adhoc-Sensor-Network/scripts
```

The git contains the following scripts nessesary for this project:

- dht11_receiver.py - python code to recieve data transmitted on the ad hoc network and print it on a display/
- dht11_sender.py - python code to collect data from a DHT11 and trasmitt data over an ad hoc network to a receiver pi.
- receiver_pi_setup.sh - bash script to intall packages and set up the ad hoc network for a receiver pi.
- receiver_pi_start.sh - bash script to intall packages and set up the ad hoc network for a sender pi.

If you need to rejoin the folder location:

```
cd ~/Raspberry-Pi-Zero-Adhoc-Sensor-Network/scripts
```


5. To ensure the code cloned from the Github repository is up to date, update the Pi's code:

```
git reset --hard  
git pull origin main
```

Once finished, go onto Section 3.3.1 to finish setting up a sender pi node or Section 3.3.2 to finish setting up a receiver pi node.

3.3.1 Setting up Sender Pi Node

Wiring the DHT11 to the Sender Pi Node Wire up the DHT11 to the Sender Pi as it needs the instrument to collect temperature and humidity data.

- Connect the VCC(+) pin of the DHT11 to the 5V pin on the Raspberry Pi.
- Connect the GND(-) pin of the DHT11 to the GND pin on the Raspberry Pi.
- Connect the DATA pin of the DHT11 to the GPIO pin on the Raspberry Pi. GPIO pin 4 was used for all Sender Pi's in this project.

Running Sender Scripts on the Raspberry Pi Once the sender pi node is properly wired up, run the following unix commands.

1. Run the following commands to give the user executable permissions on the sender bash scripts:

```
chmod +x sender_pi_setup.sh
```

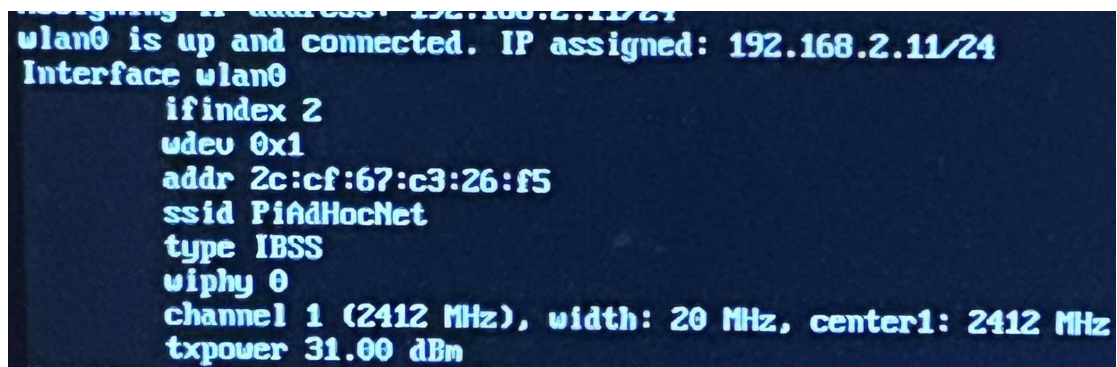
2. Run the sender_pi_setup.sh. Make sure you are connected to the internet as this script installs additional dependencies required.

```
./sender_pi_setup.sh <.x of 192.168.2.x> [GPIO pin]
```

- <.x of 192.168.2.x> is a required argument. It sets up the IP address of the Pi. Make sure it is a unique address different from other Pi's in the adhoc system
- The [GPIO pin] argument is optional if you used a different pin than GPIO 4. Leave blank if data pin of DHT11 is connected to GPIO 4.
- ex: **./sender_pi_start.sh 1** : IP address of 192.168.2.1, GPIO pin 4 (since left blank)

The sender_pi_setup.sh bash script installs a lot of libraries to set up DHT11 and multi-hop ad-hoc network.

- python3-venv and python3-pip package enables the usage of python virtual environments and the downloading of python packages within those environments to set up DHT11 data collection and communication.
- build-essential and python3-dev package allows for the compiling of python scripts within the Pi.
- libgpiod2 allows the Pi to interface the DHT11 through the GPIO pin.
- RPI.GPIO and adafruit-circuitpython-dht allows the dht11_sender.py script to run properly.
- batctl package allows for the pi to use the B.A.T.M.A.N ad hoc protocol which allows the pis to use multi-hop routing.

A terminal window with a black background and blue text. The text shows the configuration of the wlan0 interface. It starts with 'wlan0 is up and connected. IP assigned: 192.168.2.11/24'. Below this, it lists 'Interface wlan0' followed by several parameters: 'ifindex 2', 'wdev 0x1', 'addr 2c:cf:67:c3:26:f5', 'ssid PiAdHocNet', 'type IBSS', 'wiphy 0', 'channel 1 (2412 MHz), width: 20 MHz, center1: 2412 MHz', and 'txpower 31.00 dBm'.

```
reconfiguring IP address: 192.168.2.11/24
wlan0 is up and connected. IP assigned: 192.168.2.11/24
Interface wlan0
    ifindex 2
    wdev 0x1
    addr 2c:cf:67:c3:26:f5
    ssid PiAdHocNet
    type IBSS
    wiphy 0
    channel 1 (2412 MHz), width: 20 MHz, center1: 2412 MHz
    txpower 31.00 dBm
```

Figure 2: Showcasing the sender pi network up.

Once the script is run, the following should be seen on the console. If the image is seen here, that means that the DHT11 is properly working and the sender pi is pulsing data on the Ad Hoc Network.

Once all 5 sender pis have been set up, disconnect the monitor and keyboard from the sender pis and connect them to the receiver pi. The sender pi's will no longer need external peripherals as it will automatically collect and transmit environmental data.

3.3.2 Setting up Receiver Pi Node

The receiver pi node set up does not require any additional peripherals outside of a keyboard and monitor.

Running Receiver Scripts on the Raspberry Pi Once the receiver pi node is on, run the following unix commands.

1. Run the following commands to give the user executable permissions on the sender bash scripts:

```
chmod +x receiver_pi_setup.sh
```

2. Run the receiver_pi_setup.sh. Make sure you are connected to the internet as this script installs additional dependencies required.

```
./receiver_pi_setup.sh <.x of 192.168.2.x>
```

- <.x of 192.168.2.x> is a required argument. It sets up the IP address of the Pi. Make sure it is a unique address different from other Pi's in the ad hoc system.
- ex: **./sender_pi_start.sh 9** : IP address of 192.168.2.9

The receiver_pi_setup.sh bash script installs a lot of libraries to set up DHT11 and multi-hop ad-hoc network much like the sender pi.

- The main difference between the receiver and sender pi package installation is that the receiver does not install RPI.GPIO and adafruit-circuitpython-dht as the receiver pi does not need to translate the DHT11 data into ASCII characters - the sender pi does it already.
- batctl package allows for the receiver pi to use the B.A.T.M.A.N ad hoc protocol which allows the pis to use multi-hop routing.

Once the script is run, the following should be seen on the console

```
sent 5,714 bytes received 95 bytes 11,618.00 bytes/sec
total size is 10,773 speedup is 1.85
■ Setting up ad hoc Wi-Fi network...
Stopping NetworkManager...
Removed "/etc/systemd/system/dbus-org.freedesktop.nm-dispatcher.service".
Removed "/etc/systemd/system/network-online.target.wants/NetworkManager-wa
Removed "/etc/systemd/system/multi-user.target.wants/NetworkManager.service
Bringing down wlan0...
Joining ad hoc network PiAdHocNet on channel 2412...
Assigning IP address: 192.168.2.11/24
wlan0 is up and connected. IP assigned: 192.168.2.11/24
Interface wlan0
    ifindex 2
    wdev 0x1
    addr 2c:cf:67:c3:26:f5
    ssid PiAdHocNet
    type IBSS
    wiphy 0
    channel 1 (2412 MHz), width: 20 MHz, center1: 2412 MHz
    txpower 31.00 dBm
Running DHT11 receiver Python script...
Listening for DHT11 data on port 5005... (Ctrl+C to stop)
Temp: 21.6°C, Humidity: 56.0% (from 192.168.2.5)
Temp: 21.6°C, Humidity: 56.0% (from 192.168.2.5)
Temp: 21.6°C, Humidity: 56.0% (from 192.168.2.5)
Temp: 21.6°C, Humidity: 56.0% (from 192.168.2.5)
```

Figure 3: Showcasing the receiver pi network up and receiving 1 sender node.

If the image is seen here, that means that the ad hoc network is connected to the receiver pi and its properly collecting data on the network.

3.4 End Result

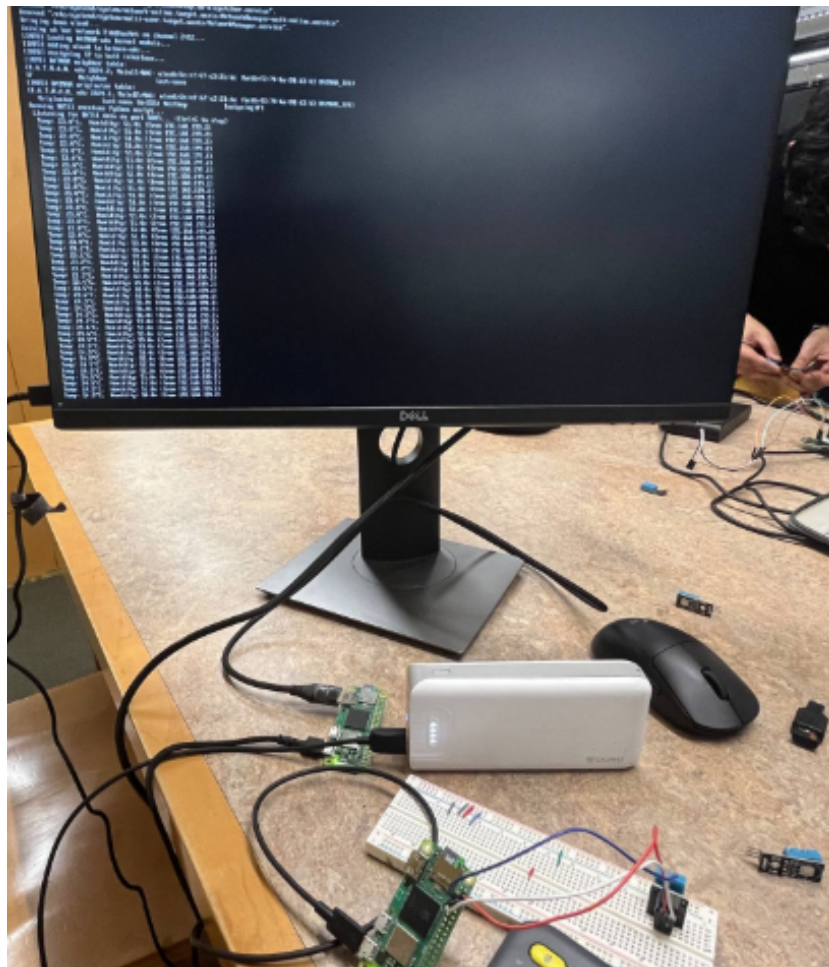


Figure 4: Receiver node receiving data from 4 other sender pis. One sensor pi with a DHT11 located directly below the receiver pi node.

For the purpose of this project, each pi was assigned an individual IP address:

- Receiver Pi - 192.168.199.1
- Sender Pi 1 - 192.168.199.2
- Sender Pi 2 - 192.168.199.3
- Sender Pi 3 - 192.168.199.4
- Sender Pi 4 - 192.168.199.5

During testing, one of the Raspberry Pi Zero 2W broke so that is why there is only 5 Pis instead of 6.

4 PROJECT RESULTS

4.1 Sensor Node & Receiver Node (Objective #1 #2)

```

Temp: 23.3°C, Humidity: 48.0% (from 192.168.199.4)
Temp: 27.1°C, Humidity: 42.0% (from 192.168.199.3)
Temp: 22.3°C, Humidity: 51.0% (from 192.168.199.2)
Temp: 24.5°C, Humidity: 45.0% (from 192.168.199.5)
Temp: 23.3°C, Humidity: 48.0% (from 192.168.199.4)
Temp: 27.2°C, Humidity: 42.0% (from 192.168.199.3)
Temp: 22.3°C, Humidity: 51.0% (from 192.168.199.2)
Temp: 24.5°C, Humidity: 45.0% (from 192.168.199.5)
Temp: 23.3°C, Humidity: 48.0% (from 192.168.199.4)
Temp: 27.2°C, Humidity: 42.0% (from 192.168.199.3)
Temp: 22.3°C, Humidity: 52.0% (from 192.168.199.2)
Temp: 24.5°C, Humidity: 45.0% (from 192.168.199.5)
Temp: 23.3°C, Humidity: 48.0% (from 192.168.199.4)
Temp: 27.2°C, Humidity: 42.0% (from 192.168.199.3)
Temp: 22.3°C, Humidity: 52.0% (from 192.168.199.2)
Temp: 24.5°C, Humidity: 45.0% (from 192.168.199.5)
Temp: 23.3°C, Humidity: 48.0% (from 192.168.199.4)
Temp: 27.3°C, Humidity: 42.0% (from 192.168.199.3)
Temp: 22.3°C, Humidity: 52.0% (from 192.168.199.2)
Temp: 24.5°C, Humidity: 45.0% (from 192.168.199.5)
Temp: 23.3°C, Humidity: 48.0% (from 192.168.199.4)
Temp: 27.3°C, Humidity: 42.0% (from 192.168.199.3)
Temp: 22.3°C, Humidity: 52.0% (from 192.168.199.2)
Temp: 24.5°C, Humidity: 45.0% (from 192.168.199.5)
Temp: 23.3°C, Humidity: 48.0% (from 192.168.199.4)
Temp: 27.3°C, Humidity: 41.0% (from 192.168.199.3)
Temp: 22.2°C, Humidity: 52.0% (from 192.168.199.2)
Temp: 24.5°C, Humidity: 45.0% (from 192.168.199.5)
Temp: 23.3°C, Humidity: 48.0% (from 192.168.199.4)
Temp: 27.4°C, Humidity: 41.0% (from 192.168.199.3)
Temp: 22.2°C, Humidity: 52.0% (from 192.168.199.2)
Temp: 24.5°C, Humidity: 45.0% (from 192.168.199.5)
Temp: 27.4°C, Humidity: 41.0% (from 192.168.199.3)
Temp: 24.5°C, Humidity: 45.0% (from 192.168.199.5)
Temp: 23.3°C, Humidity: 48.0% (from 192.168.199.4)
Temp: 22.2°C, Humidity: 52.0% (from 192.168.199.2)
Temp: 27.4°C, Humidity: 41.0% (from 192.168.199.3)
Temp: 24.5°C, Humidity: 46.0% (from 192.168.199.5)
Temp: 23.3°C, Humidity: 48.0% (from 192.168.199.4)
Temp: 22.2°C, Humidity: 52.0% (from 192.168.199.2)
Temp: 24.5°C, Humidity: 46.0% (from 192.168.199.5)
Temp: 23.3°C, Humidity: 48.0% (from 192.168.199.4)
Temp: 27.5°C, Humidity: 41.0% (from 192.168.199.3)
Temp: 22.2°C, Humidity: 52.0% (from 192.168.199.2)
Temp: 24.5°C, Humidity: 46.0% (from 192.168.199.5)
Temp: 23.3°C, Humidity: 48.0% (from 192.168.199.4)

```

Figure 5: The Receiver Node receiving data from 4 unique sender pis.

The receiver node properly senses transmitted data on the ad hoc network being sent out by the sender pi nodes and displays the ASCII characters transmitted onto the display. This fulfills objective 1 as the sender nodes are properly measuring the temperature and humidity and sending the data over an ad hoc network. This also fulfills objective 2 as the receiver node is receiving the data over an ad hoc network and displaying it.

4.2 Multi-hop Routing (Objective #3)



No Nodes between Sender and Receiver



1 Node (at the Red Circle) between the Sender and Receiver

As seen in the figure to the left - assuming at the receiver node is located where the picture was taken - the distance between the sender and receiver is about 1/3 of Cal Poly Pomona's library. However, by placing a node at the red circle as seen on the picture to the right between the sender and receiver, the distance that the sender node can send data to the receiver node is greatly increased. The max distance found was the entire length of the 3rd floor of the Cal Poly Pomona library because humans cannot phase through walls.

This is able to happen because B.A.T.M.A.N ad hoc protocol allows for multi-hop by sensing the best connection between nodes.

This fulfills objective 3 as multi-hop ad hoc routing was implemented into the system as seen with the increased distance between the sending and receiver nodes as long as there are additional nodes between the two.

4.3 Battery Life Efficiency (Objective #4)

The Raspberry Pi Zero 2W consumes 1.3 Watts per hour. Enabling ad-hoc on the Pi consumes an additional 0.3 watts per hour. The DHT11 sensor consumes negligible power.

$$\begin{aligned} P_{total} &= P_{pi} + P_{ad-hoc} = 1.3W + 0.3W \\ &= 1.6 \text{ watt-hour} \end{aligned}$$

A Raspberry Pi 2W requires 5V to be functional. Because portable batteries are measured in milliampere hours and the objective 4 states that a node be online for at least 5 hours:

$$\begin{aligned} \text{Minimum mAh} &= \frac{P_{total}}{5V} \times 5 = \frac{1.6Wh}{5V} \times 5 \\ &= 1.6 \text{ amps per 5 hours} \\ &= 1600 \text{ milliamperes per 5 hours} \end{aligned}$$

As the cheapest batteries on Amazon or any online retailer or warehouse can carry well over 10,000mAh for less than \$10, it can be safely assumed that most if not all portable batteries with a USB-A output will fulfill objective 4 because each sender node is incredibly power efficient.

Do keep in mind that this excludes the receiver node as powering a display takes exponentially more energy to power than the actual Pi itself.

5 SUMMARY OF CHALLENGES AND HOW THEY WERE SOLVED

5.1 Switching from IBSS Networking to B.A.T.M.A.N

In the initial phase, the team configured the three Raspberry Pi Zero 2W nodes into an ad-hoc (IBSS) WLAN. In IBSS mode each node joins a common SSID without an access point. However, this simple configuration showed severe limitations. All devices had to remain within direct radio range of each other, since “IBSS has no provision for relaying packets, so all nodes must be in radio range of all other nodes”.

In practice we found that more than about three Pi’s could not maintain stable connectivity – adding a fourth device often caused links to fail. Moreover, IBSS mode on the Pi Zero 2W cannot use WPA2 encryption (the onboard Wi-Fi only supports open IBSS), forcing an unprotected network. In sum, our IBSS mesh was flat and static: it lacked any dynamic routing or failover. If a node moved or went offline, there was no automatic re-routing the network simply split. Traditional IEEE 802.11 IBSS networks were known to have “limitations (no always-on discovery, no power-saving coordination, weak security)”, reflecting the reality we observed.

These limitations directly impacted our objectives. The mesh was not scalable or resilient: it could not grow beyond a few nodes, and it could not route around failures. Test deployments showed that if any Pi rebooted or lost power, the rest became isolated. We also could not automatically integrate new nodes on the fly, since there was no discovery or routing protocol. In short, the IBSS solution fell short of the project’s aims (a flexible, self-healing mesh) and could not support a larger network.

To overcome these issues, the team adopted the BATMAN-ADV (B.A.T.M.A.N. – “Better Approach To Mobile Ad-hoc Networking – Advanced”) routing protocol. BATMAN-ADV is specifically designed for multi-hop wireless meshes: it provides decentralized, dynamic routing across arbitrary topologies. In BATMAN each node broadcasts small “originator” messages with sequence numbers; neighbors rebroadcast them, and the originating node detects paths when it hears its own message returned via another node. Crucially, no single node maintains the entire network map; each router only keeps the next-hop information toward each destination.

This means the mesh can automatically form routes without central coordination, and packets follow dynamically chosen paths based on recent link quality. In practice, this yields a self-healing, multi-path network: if one link goes down, the protocol simply favors alternate neighbors, so traffic “finds its way” through the best available route.

5.2 Public WiFi vs Private WiFi for setting up Pis

During the early stages of the project, we encountered significant challenges when attempting to configure and update the Raspberry Pi Zero 2W devices using public Wi-Fi networks, specifically our university's "CPP Guest" network. Although this network provided basic internet access, it was not suitable for development tasks like cloning repositories, installing packages, or performing system updates. The main issue stemmed from network-level authentication and captive portal restrictions, which blocked the Raspberry Pis from properly establishing outbound HTTPS connections. As a result, commands like `git clone` or `apt update` would fail or hang indefinitely due to the device being unable to complete authentication required by the network.

Public or guest Wi-Fi networks often employ enterprise-grade firewalls, proxy servers, or captive portals that require a browser-based login or device registration—none of which are practical (or sometimes even possible) on headless Raspberry Pi setups. Furthermore, such networks may restrict or throttle traffic types that appear suspicious or come from unregistered devices, adding further complications when working with SSH, Git, or package managers.

To work around these limitations, we transitioned to using private home Wi-Fi networks and mobile phone hotspots for all setup and development activities. These networks allowed for full, unrestricted internet access without the need for login portals, making it possible to clone repositories from GitHub, download software dependencies, and configure system packages directly on the Pi devices. Using personal hotspots also gave us more control over the network environment during testing, allowing the devices to communicate freely without interference from external security policies.

This experience highlighted a common issue in IoT and edge computing development: public Wi-Fi networks are often incompatible with device provisioning workflows. For future deployments, especially in institutional settings, we recommend ensuring access to a private network or using a temporary hotspot during initial setup and configuration phases.

5.3 Saving Costs

A key logistical and financial challenge during the early stages of the project was the large quantity of components required to build and manage a functional Raspberry Pi mesh network. Our initial prototype plan involved deploying approximately six Raspberry Pi Zero 2 W units, each requiring a dedicated power cable, HDMI cable, DHT11 temperature/humidity sensor, and temporary peripherals such as keyboards and mice for setup. This resulted in the need for: 6 Raspberry Pi Zero 2 Ws, 6 micro-USB power cables, 6 HDMI cables, 5 DHT11 sensors, multiple keyboards and mice, and at least one monitor per unit for setup.

This hardware footprint introduced several issues. The sheer number of cables and components created significant clutter and confusion, especially when connecting and disconnecting devices during testing. More importantly, the setup and teardown process became time-consuming and error-prone, making rapid iteration and testing difficult. Additionally, limited access to monitors and input devices further constrained our ability to efficiently configure each node.

To address these concerns, we adopted a minimalist, cost-saving approach centered around reducing hardware redundancy and optimizing workflow. The key solution was to utilize the lowest number of components possible without compromising functionality. Rather than fully equipping each Raspberry Pi with its own peripherals, we strategically divided the Raspberry Pi units among group members and focused only on configuring and testing a few nodes at a time.

This decision had several benefits: resource sharing: With only one or two monitors available, team members took turns verifying that each Raspberry Pi was functioning properly before integrating it into the network. This allowed the team to avoid purchasing extra displays or keyboards, significantly reducing project costs.

Remote access via SSH: By configuring SSH access on the Raspberry Pis early in the setup process, we eliminated the need for separate input devices for each unit. After the initial setup, all subsequent management could be done remotely from a laptop or desktop. Parallel development: Distributing the Pi units across team members enabled each person to test and refine specific configurations or scenarios individually. When needed, nodes were brought together to test more complex, multi-hop mesh topologies.

Overall, this frugal and efficient hardware strategy allowed us to maximize progress with limited resources, avoiding unnecessary purchases and minimizing hardware clutter. This approach not only conserved time and money, but also laid the foundation for scalable deployment practices, especially valuable for field setups like those envisioned in greenhouse environments.

6 CONCLUSIONS

In conclusion, the B.A.T.M.A.N. (Better Approach To Mobile Ad hoc Networking) protocol, developed by the Freifunk community, a noncommercial grassroots initiative promoting free and decentralized network infrastructure in Germany, offers a robust solution for implementing multi-hop mobile ad hoc networks. Its architecture allows nodes to dynamically discover optimal paths without relying on centralized routing tables, making it particularly well-suited for decentralized, scalable deployments.

One practical application is in smart agriculture, such as deploying Raspberry Pi Zero devices equipped with DHT11 temperature and humidity sensors within a greenhouse. These low-cost devices can communicate across large or elongated spaces using B.A.T.M.A.N.'s multi-hop capability, ensuring sensor data reaches a central controller or alert system even if direct line-of-sight communication is unavailable. Moreover, Raspberry Pis can be configured to trigger alerts when sensor readings cross predefined thresholds, enabling proactive environmental control and improving plant health outcomes.

Beyond agricultural uses, the flexibility of ad hoc networks allows them to integrate other sensing modules, such as ultrasonic sensors, to create a low-cost radar-like system for defense or military scenarios. Such systems could be used for perimeter monitoring, detecting intrusions, or tracking movement without relying on fixed infrastructure. Similarly, the same decentralized communication model can be applied to a range of monitoring applications, from measuring wind speeds across wide areas to tracking vital signs in wearable health monitoring systems for individuals.

The key strength of B.A.T.M.A.N. lies in its decentralized routing mechanism. Unlike the Optimized Link State Routing (OLSR) protocol it was designed to replace, B.A.T.M.A.N. does not require each node to have a complete map of the network. Instead, it distributes routing decisions across the network, making it more resilient and better suited for dynamic or large-scale mesh topologies. This makes B.A.T.M.A.N. a powerful enabler for creating reliable, self-healing communication networks across a wide array of disciplines and use cases.

7 REFERENCES

[1] J. Yi, A. Krishnamurthy, and M. Faloutsos, "Centrality-based routing in delay tolerant networks," *IEEE Network*, vol. 27, no. 2, pp. 30–35, Mar.–Apr. 2013. doi: 10.1109/MNET.2013.6470393.

[2] R. M. Martínez, G. D. R. Ordoñez, A. J. O. Martínez, and M. R. Pérez, "Experimental analysis of multi-hop communication protocols for wireless mesh networks in smart city applications," in *2023 IEEE International Conference on Smart Cities and Green ICT Systems (SMARTGREENS)*, Prague, Czech Republic, Apr. 2023, pp. 1–8. doi: 10.1109/SMARTGREENS60104.2023.10339567.

[3] Raspberry Pi Stack Exchange, "Questions tagged [raspberrypi-zero-w]," [Online]. Available: <https://raspberrypi.stackexchange.com>. [Accessed: May 7, 2025].

[4] Wikipedia, "B.A.T.M.A.N. (routing protocol)," [Online]. Available: <https://en.wikipedia.org/wiki/B.A.T.M.A.N.> [Accessed: May 7, 2025].

[5] Open-Mesh, "BATMAN-adv quick start guide," [Online]. Available: <https://www.open-mesh.org/projects/batman-adv/wiki/Quick-start-guide>. [Accessed: May 7, 2025].